Sleep Apnea Detection System - Complete Demo Guide

Sleep Apnea Detection System - Complete Demo Guide

Project Title: AI-Powered Sleep Apnea Detection and Monitoring Using Deep Learning on ECG Signals

Team Members: Irin Babu (FIT23CSD033) | Mariya Jiji (FIT23CSD040) | Neha B S (FIT23CSD044) | Nisha P R (FIT23CSD047)

Guide: Simi Stephen, AP, CSE

1. About The Project

This project implements an explainable Sleep Apnea Detection system using ECG signals with a hybrid deep learning architecture (CNN-BiLSTM + Attention mechanism). The system classifies sleep apnea events from ECG recordings and provides interpretable results.

2. Dataset Used

Primary Dataset: PhysioNet Apnea-ECG Database v1.0.0

- **Source:** https://physionet.org/content/apnea-ecg/1.0.0/
- **Type:** Real clinical ECG recordings from patients with suspected sleep apnea
- **Format:** WFDB format (.hea, .dat, .apn annotation files)
- **Sampling Rate:** 100 Hz
- **Annotations:** Expert-scored apnea events (A = Apnea, N = Normal)
- **Note:** All data used is from the original PhysioNet Apnea-ECG database - no synthetic datasets were used. The model has been trained on authentic clinical recordings from this database.

3. Model Architecture

Model Name: SleepApneaFusion (CNN-BiLSTM + Attention)

Components:

1. **Spectrogram Branch (ResNet18)** - Converts ECG to STFT spectrogram, uses ResNet18 backbone for spatial feature extraction with Grad-CAM explainability

2. **RR Interval Branch (1D-CNN + BiLSTM + Attention)** - Extracts RR intervals from ECG, processes with CNN layers, BiLSTM for temporal dependencies, and attention mechanism for important beat weighting

3. **SpO2 Branch (MLP)** - Processes SpO2-derived statistical features (min, max, mean, std, skewness, range, desaturations)

Fusion:

- Features concatenated (256 + 256 + 16 = 528 dimensions)
- Fully connected layers with dropout for final classification
- Binary classification: Normal vs Apnea Detected

Explainability:

- Grad-CAM for spectrogram visualization
- Attention weights for RR sequence importance

4. Setup Instructions

Step 1: Clone the Repository

git clone https://github.com/ananthakrishnan754/sleep-apnea-detection-cli.git
cd sleep-apnea-detection-cli

Step 2: Install Dependencies

pip install -r requirements.txt

Step 3: Verify Model Weights

The trained model weights are in `weights/fusion_best.pth`

5. Running Predictions (Command Line)

Method 1: Predict from ECG file (.npy)

python cli/predict_file.py --input data/processed/X.npy

Method 2: Predict from ECG text file

python cli/predict_file.py --input your_ecg_data.txt

Method 3: Predict from JSON file

python cli/predict_file.py --input your_ecg_data.json

Method 4: Include SpO2 data

python cli/predict_file.py --input your_ecg_data.npy --spo2 "95,94,93,95,96"
Or provide SpO2 from file:
python cli/predict_file.py --input your_ecg_data.npy --spo2 spo2_data.txt

Expected Output:

==========
SLEEP APNEA DETECTION - MODEL PREDICTION
==========
Loading trained model...
Model loaded from weights/fusion_best.pth
==========
Loaded ECG: XXXXX samples

PREDICTION RESULT

Prediction: APNEA DETECTED / NORMAL
Confidence: XX.XX%
AHI Proxy: XX.X events/hour
Severity: Normal/Mild/Moderate/Severe

[MODEL] CNN-BiLSTM + Attention | Spectrogram(ResNet18) + RR(BiLSTM) + SpO2(MLP)
==========

Result saved to prediction_result.json

6. Alternative Demo Methods

CLI Demo (Simulated)

python cli/run_demo.py --mode watch_sim

Calibration

python cli/calibrate_user.py --user --hr_json

7. Model Files Location

- **Trained Weights:** `weights/fusion_best.pth`
- **Model Architecture:** `models/fusion_model.py`
- **Preprocessing:** `preprocessing/stft_spectro.py`, `preprocessing/rr_extract.py`, `preprocessing/spo2_prep.py`
- **Prediction Script:** `cli/predict_file.py`

8. Important Notes

- All predictions use the trained model loaded from `weights/fusion_best.pth`
- The model was trained on real clinical data from PhysioNet Apnea-ECG database
- No synthetic/fake datasets were used for training
- Results are saved to `prediction_result.json` after each prediction
- For GPU acceleration, ensure CUDA is available (falls back to CPU automatically)

9. Quick Test Command

After cloning and installing dependencies, run this single command to test:

```
python cli/predict_file.py --input data/processed/X.npy
```